Hybrid Agentic Reasoning for ARC-AGI via a Constrained DSL Simulator

Author A Author B Author C Author D Author E
Department of Computer Science and Engineering
University of California, San Diego
La Jolla, CA 92093
{authora, authorb, authorc, authord, authore}@ucsd.edu

Abstract

The ARC-AGI benchmark evaluates abstract reasoning through grid transformation puzzles. Existing methods often sit at two extremes: symbolic search over domain-specific languages (DSLs), which is reliable but expensive, and direct large language model (LLM) inference, which is flexible but brittle due to code-generation and execution failures. We present a text-adventure-style ARC simulator where agents solve puzzles by issuing predefined DSL actions with environment feedback. This interface constrains the action space, removes free-form coding errors, and supports direct comparison across agent families.

Using this common environment, we evaluate three paradigms. API-based LLM agents perform strongly in a minimal-DSL setting, with gpt-oss-120b solving 82/85 puzzles and kimi-k2.5 solving 73/85. In contrast, supervised fine-tuning (SFT) on Qwen3-1.7B from 582 solver-replay pairs fails to solve any of 27 evaluation puzzles and reaches only about 6% cell accuracy. Replacing these with gpt-oss-120b-generated solution trajectories for distillation improves cell accuracy to roughly 15%, but the student still solves no puzzles; we offer a preliminary discussion and leave a full explanation to future work. Reinforcement learning (RL) with GRPO also fails as a standalone solver under sparse rewards, but the learned policy becomes effective as an operator-ranking prior for beam search, matching DFS solved count while reducing runtime.

These results suggest that ARC-AGI is best approached as a hybrid systems problem: constrained symbolic interfaces and explicit verification provide reliability, while learned priors improve search efficiency.

1 Introduction

ARC (Abstraction and Reasoning Corpus) was designed as a test of fluid, compositional intelligence under low-data supervision [Chollet, 2019]. In each puzzle, a system must infer a latent transformation from a few input-output grid examples and apply it to a held-out input. The key difficulty is not pattern matching alone, but discovering and composing reusable operations.

Two dominant approaches are generally adopted to solve these ARC puzzles. The most common one is to define a DSL and search programs with DFS/BFS/A* over that space, as ARC is a pure grid-transformation problem [Hodel, 2023, 2024]. However, this approach is costly as the time and efforts scale up exponentially with the steps of DSLs needed for a specific puzzle. A second strategy is to query LLM APIs directly, either querying directly for the final output grid or asking for a helper code snippet that transforms the test input grid to the test output. While this route is flexible, it often brittle due to malformed code, execution errors, and shallow pattern matching [Wang et al., 2023, Ho et al., 2025, Lee et al., 2023]. This motivates our core question: can we combine the structural reliability of DSL execution with the adaptive planning of modern language agents?

Preprint.

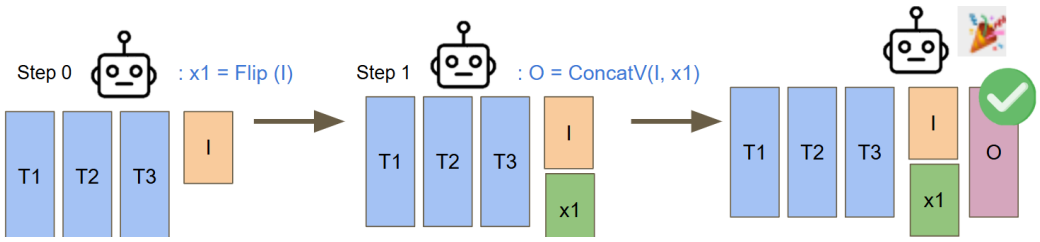

Figure 1: **Overview of the ARC interaction environment.** The agent starts with the three training input-output pairs and the test input in its inventory. At each step, it selects a valid DSL action, the environment deterministically executes that action, appends the resulting intermediate artifact to the inventory, and returns the updated state. This enables step-by-step program construction while avoiding free-form implementation errors.

Inspired by this, we propose a text-adventure-style simulator environment where solving ARC puzzles is cast as sequential decision making over constrained DSL actions. Instead of writing arbitrary Python snippets, the AI agent is required to select one executable DSL operation at a time, receives updated state feedback, and constructs a candidate program incrementally. This controlled setup enables a side-by-side comparison of API LLM agents, imitation learning, and RL under a shared interface.

Implementing and experimenting with this simulator-style environment with diverse kinds of AI agents, our main contributions, in summary, are:

- We formulate ARC solving as interactive tool use in a constrained DSL simulator.
- We provide a unified empirical comparison of API LLM, SFT, and RL methods using reported experimental logs.
- We show that sparse-reward RL is weak as an end-to-end solver but useful as a search prior.
- We document concrete failure modes such as hardcoded outputs and interface violations.

2 Related Work

ARC and symbolic program induction. ARC has become a central testbed for abstraction-focused reasoning [Chollet, 2019]. Since tasks are exact grid transformations, DSL-based symbolic approaches are a natural fit: define a transformation language and search for valid programs, often with DFS/BFS/A* style procedures [Hodel, 2023, 2024]. Recent benchmark work also extends ARC with human baselines and broader evaluation methodology [Chollet et al., 2024, Le Gris et al., 2025].

LLMs on ARC-like reasoning. Another line of work uses direct LLM querying. Typical prompts ask the model to infer the output grid from a few input-output examples, or to emit helper code (e.g., a `solve(...)` function) for the test case. Prompting and API-based reasoning methods can be powerful, but they also reveal failure modes in compositional and tool-grounded settings [Wang et al., 2023, Lee et al., 2023, Tan et al., 2024, Ho et al., 2025].

Language agents, imitation learning, and RL. Tool-using language agents have shown strong performance when actions are externally verifiable [Yao et al., 2023, Schick et al., 2023]. Behavior cloning is a standard approach for offline policy learning but depends heavily on support coverage. In RL, sparse-reward settings are known to be difficult [Sutton and Barto, 2018, Schulman et al., 2017]; recent policy optimization variants such as GRPO highlight both opportunities and stability constraints [Shao et al., 2024].

3 ARC Puzzles

An ARC puzzle consists of three training input-output pairs, $\{(X_i, Y_i)\}_{i=1}^{3}$, together with a test input X_{test}. The goal is to infer the latent transformation rule shared by the training examples and apply it to produce the correct test output Y_{test}.

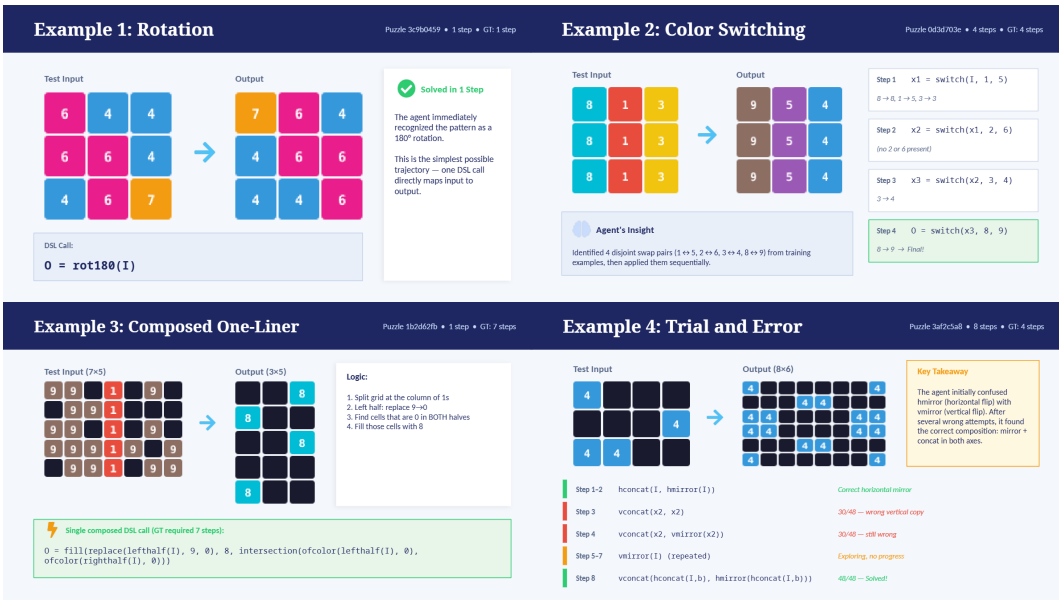

Figure 2: **Qualitative Examples of ARC Solution Trajectories.** Representative puzzles and their corresponding solution paths from GPT-oss-120b, highlighting the diversity of reasoning patterns supported by our environment and the agent's trial history.

Rather than treating it as a one-shot prediction, we formulate solving the ARC puzzles as sequential decision making in a constrained DSL environment, motivated by the limitations of two common alternatives that pure DSL search is often accurate but computationally expensive, and that direct LLM generation is flexible but can fail due to long-horizon reasoning errors or incorrect code execution. Our formulation aims to combine symbolic reliability with agent-based decision making.

Formally, at each step t, the agent interacts with an environment via:

- **State** s_t: a serialized inventory $\mathcal{V}_t$ containing the three training pairs, the test input, and all intermediate puzzle grids generated by the agent itself.
- **Action** a_t: a valid DSL command applied to existing entries in $\mathcal{V}_t$ (e.g., transform, extract, compose), producing a new artifact.
- **Transition**: deterministic execution of the selected DSL command by the environment, followed by insertion of the resulting artifact into $\mathcal{V}_{t+1}$.
- **Termination**: the episode ends when the agent submits an output grid; success is achieved if it exactly matches the target grid.

This turns one-shot prediction into constrained program construction. Compared with solving the ARC puzzles via one-shot, having this simulator environment in the DSL space guarantees the exclusion of code execution errors when the agent tries to operate on the grids. By requiring the agent to focus on outputting the DSL actions instead of coding, we therefore force it to focus more on the abstract logic reasoning.

4 Environment and Methods

4.1 Text-adventure DSL simulator

As the first contribution, we built up a simulator to enable the AI agents to play with the ARC puzzles in the DSL space. As stated in Sec. 3, at each step, the agent is provided with the three training examples, the test input, and the current inventory of intermediate artifacts to the agent as the observation. Then, the agent is required to output one valid DSL action, and the environment executes it using predefined off-the-shelf DSL operators. The resulting intermediate puzzle grid is appended to the inventory, and the agent receives an updated observation describing the new state.

Similar to playing a text-adventure game, the agent does not write an entire solver program at once, but instead incrementally constructs a solution through a sequence of executable actions. For example, starting from the initial inventory $\{(X_i, Y_i)\}_{i=1}^{3} \cup \{X_{\text{test}}\}$, the agent may generate an intermediate variable such as $x_1 = \texttt{Flip}(X_{\text{test}})$, after which the environment updates the inventory to include x_1. Subsequent actions may then reuse both the original inputs and newly created artifacts. This setup avoids the brittleness of free-form code generation, since every action must be chosen from a predefined DSL and is executed deterministically by the environment. Additionally, it naturally supports multi-step reasoning, since the agent can inspect intermediate results and adapt its future actions accordingly. As illustrated in Fig. 1, the same environment can be used by both API-based LLM agents and offline-trained agents such as SFT or RL policies.

We also demonstrate more examples of puzzle-solving trajectories from GPT-oss-120b in Fig. 2.

4.2 API-based LLM agents

API agents first plan over puzzle structure and then execute one DSL action per step with feedback. We evaluate two settings:

- **Minimal DSL:** only necessary operators are exposed.
- **Distractor DSL:** irrelevant operators are added to probe robustness.

Minimal DSL. In this setting the agent receives only the DSL operators used by the ground-truth solution (typically 3–7 out of 160). Each episode begins with a planning round in which the LLM analyzes the training pairs, followed by iterative execution: one DSL call per turn with environment feedback (success/error message and current variable state). The episode ends when the output grid matches ground truth or a 20-step budget is exhausted. We evaluate five models on 85 puzzles (46 skipped for grids exceeding 10×10).

Distractor DSL.

4.3 Supervised fine-tuning (SFT)

We fine-tune Qwen3-1.7B with behavioral cloning on (observation, action) pairs: observations are formatted as prompts (puzzle id, budget, training examples, test input, last result), and actions are serialized to single-line DSL commands via `action_to_command()` (e.g., `execute hupscale(I, 3) -> x1`, `submit 0`). Expert trajectories come from either replaying programmatic solvers in the simulator or from gpt-oss-120b interacting step-by-step; we use the same prompt/action format at train and inference, with a grid search over temperature, learning rate, and epochs to select the best setup.

With 582 solver-replay pairs, SFT reaches 0 solved and $\sim$6% cell accuracy on 27 evaluation puzzles—we attribute this to data scarcity, combinatorial coverage over 160+ DSL operators, and policy collapse to short templates. Switching to gpt-oss-120b-generated trajectories improves cell accuracy to $\sim$15% but still yields 0 solved. Trajectory inspection suggests the student often learns the *format* of expert actions (valid syntax, turn-taking) rather than a policy that conditions on observation: e.g., it repeats the same first-step template across different puzzles instead of choosing puzzle-specific operators. A full explanation (capacity, distribution shift, or composition failure) is left to future work.

4.4 Reinforcement learning (GRPO)

MDP formulation. We frame ARC solving as a finite-horizon Markov Decision Process. The state s_t is a 110-dim vector encoding the agent's current inventory: color histograms for the test input and one training pair (30 dims), 5×5 downsampled grids (75 dims), and scalar features including steps remaining and a live training-accuracy probe (5 dims).

The action space includes $|\mathcal{A}|$ discrete DSL calls: unary grid transforms (e.g., `hmirror`, `rot90`), meta-operations (e.g., `objects`), binary compositions over `I` and `last_var`, parametric scale, `ofcolor`, and pairwise color `replace`. The baseline has 87 actions, later expanded to 203. A special SUBMIT action ends the episode; `last_var` references are masked at chain depth zero.

Table 1: Minimal-DSL API agent performance (85 puzzles, 20-step budget).

Model	Solved	Accuracy (%)
gpt-oss-120b	82/85	96.5
kimi-k2.5	73/85	85.9
deepseek-r1	57/85	67.1
minimax-m2	45/85	52.9
mistral-small-3.2-2506	28/85	32.9

Rewards are sparse: $r = +1.0$ for exact matches, $r = 0.05 \cdot \text{acc}_{\text{px}}$ for partial credit, and $r = -0.3$ for submitting the unmodified input. Non-terminal steps incur a -0.01 penalty.

Policy architecture. The policy π_θ is a 3-layer MLP: a linear projection from 110→128, two hidden layers of 128 with LayerNorm + ReLU, and a linear output of width $|\mathcal{A}|$, totaling 50K parameters.

GRPO training. We use Group Relative Policy Optimization [Shao et al., 2024]. Each step samples a puzzle from 104 training examples and executes $K = 8$ rollouts. Advantages are normalized within the group: $\hat{A}_k = (r_k - \mu_r)/(\sigma_r + \epsilon)$, with $\epsilon = 10^{-8}$. An entropy bonus is annealed from $\lambda_H = 0.10$ to 0.02 to encourage exploration early and consolidation later. Optimizer is AdamW, lr=3×10^{-4}, cosine decay.

Pure RL policy (Attempt 1). We train from scratch for 5,000 episodes (87-action space) without behavioral priors.

RL-guided beam search (Attempt 2). We start from the GRPO checkpoint of Attempt 1 and add beam search. Policy logits guide expansion: only the top-$B = 10$ actions are considered per node, maintaining $K = 8$ beams to a max depth of 4 DSL operations. Candidate programs are verified against all three training pairs; failed programs are discarded. Here, the policy acts as a heuristic to prune the search tree, not an autonomous solver.

5 Experiments and Results

5.1 API LLM results in minimal DSL setting

Table 1 summarizes main results on 85 evaluation puzzles. Strong frontier models solve most tasks when action semantics are constrained by DSL execution. GPT-OSS-120B achieves 96.5%, with solved puzzles requiring an average of 4.0 steps, closely matching the ground-truth average of 4.1. All five models exhaust the full 20-step budget on every failed puzzle, suggesting that agents either converge quickly or not at all. Notably, DeepSeek-R1's reasoning traces do not translate into higher accuracy; its chain-of-thought outputs frequently violate the single-command format, wasting steps on parse errors.

5.2 Behavioral pathologies and restriction ablations

The interactive environment exposes two recurrent failure modes. **(i) Hardcoding:** after repeated execution errors, the agent abandons DSL composition and emits a raw grid literal (e.g. `O = [[0,0,...],...]`). This occasionally produces a correct output on small puzzles but reflects memorization of training examples rather than understanding of the transformation rule. **(ii) Breaking character:** when the agent cannot identify the correct DSL call, it falls back to general Python, lambda expressions, list comprehensions, and manual index arithmetic, which the sandboxed executor rejects, trapping the agent in a retry loop until the step budget is exhausted.

Prompt-level instructions ("use only DSL functions", "do not hardcode") reduce but do not eliminate these behaviors: unrestricted GPT-OSS-120B solves 82/85, a single restriction yields 79/85, and combined restrictions recover to 80/85. This suggests that behavioral compliance is sensitive to

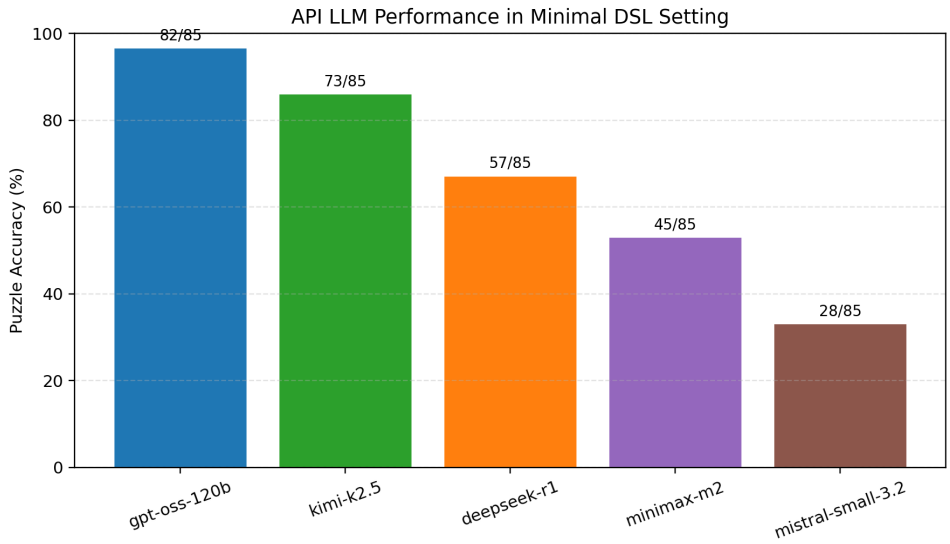

Figure 3: API LLM accuracy under minimal DSL exposure.

Table 2: Distractor ablation (solved puzzles out of 85).

Model	0 distractors	5 distractors	10/20 distractors
gpt-oss-120b	82	78	80 (10)
kimi-k2.5	73	67	68 (10)
llama-4-scout	27	–	22 (10), 19 (20)

prompt phrasing and that execution-level guardrails may be more reliable than instruction-level ones for enforcing tool discipline.

5.3 Distractor DSL robustness

Table 2 and Figure 4 show accuracy under increasing irrelevant operators. Strong models degrade less, indicating better internal filtering over expanded action spaces.

5.4 SFT and RL on 27-puzzle evaluation set

Metrics. **Cell accuracy** is the mean over episodes of (correct cells / total cells) in the submitted output grid; **solved** requires an exact match (100% cell accuracy). So non-zero cell accuracy with 0 solved is possible when no full grid is correct.

Table 3 highlights the gap between standalone learning and hybrid search. SFT fails to solve any puzzle despite tuning. Pure RL suffers from sparse-reward collapse. RL-guided beam search matches DFS solved count while reducing runtime from 2.6s to 0.4s.

Pure RL. Training GRPO from scratch yields a policy that solves 1/27 evaluation puzzles (3.7%) with average cell accuracy 21.7%. Qualitatively, it collapses to a few simple single-step programs—mostly `rot90` or `hmirror` on `I`—that happen to solve the easiest training puzzles. The first 500 episodes show no progress: all rollouts fail, reward std $\sigma_r \approx 0$, normalized advantages are undefined, and gradients are skipped. Improvement only begins when the policy stumbles on trivially solvable puzzles, after which it overfits and fails to generalize.

RL-guided beam search. Using beam search at inference boosts performance to 4/27 evaluation puzzles (14.8%) with average cell accuracy 49.5%, matching the DFS baseline while reducing evaluation runtime from 2.6s to 0.4s—a 6× speedup. Even a poorly converged policy provides

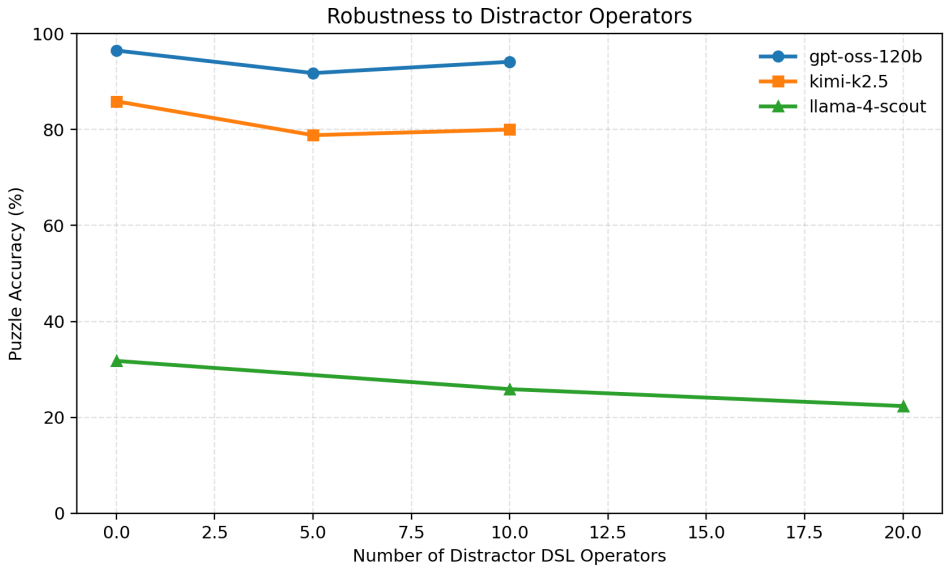

Figure 4: Accuracy under increasing distractor operators.

Table 3: SFT and RL-family results (27 puzzles).

Method	Solved	Cell Accuracy (%)	Avg Runtime (s)
SFT (solver trajectories)	0/27	6.3	–
SFT (gpt-oss-120b teacher)	0/27	∼15	–
Pure RL (GRPO)	1/27	21.7	–
RL-guided Beam Search	4/27	49.5	0.4
DFS Baseline	4/27	–	2.6

a useful inductive bias: it assigns higher logit mass to frequently appearing transformations (e.g., rotations, reflections), pruning the beam toward likely operators without solving puzzles outright. This mirrors AlphaGo [Silver et al., 2016]: the policy doesn't find the winning move but guides the search to promising branches.

Two independent beam search runs give slightly different results (3–5 solved on training, 3–4 on evaluation) due to stochastic sampling. With only 27 evaluation puzzles, a single-puzzle difference (3.7%) is expected.

6 Analysis

Why API LLMs dominate. Stronger API models combine broad pretrained priors, better in-context adaptation, and iterative correction from environment feedback. Under constrained execution, model capability becomes the dominant variable: GPT-OSS-120B solves puzzles in 4.0 steps on average versus 5.8 for Mistral Small, despite both receiving identical DSL function sets and environment feedback. Weaker models tend to repeat failing calls or oscillate between alternative formulations without making progress, consuming their entire 20-step budget. In contrast, top-performing models recover from errors within 1–2 steps—for instance, correcting a variable-ordering mistake (referencing `x3` before defining it) and immediately re-issuing the correct sequence. The planning round further separates model tiers: GPT-OSS and Kimi produce concise, actionable plans that map directly to DSL calls, while weaker models generate lengthy but unfocused analyses that fail to translate into correct execution.

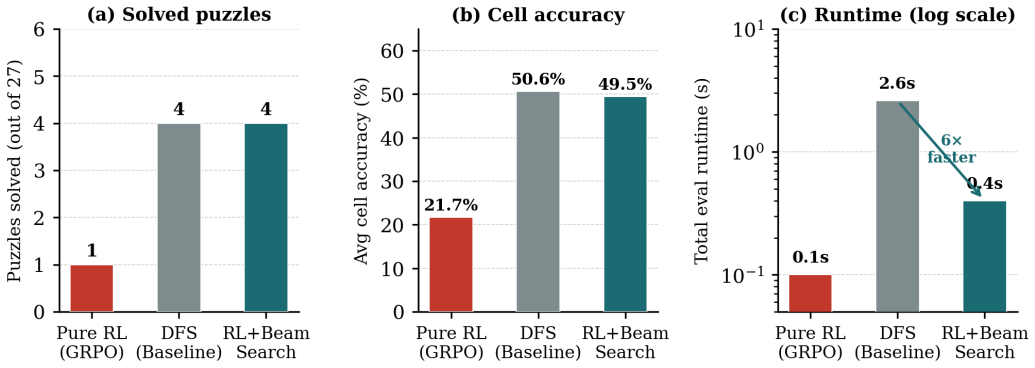

Figure 5: RL as standalone solver vs. search prior.

Why SFT collapses. Sparse demonstrations over a large DSL lead to template memorization; distillation to $\sim$15% cell accuracy still yields 0 solved, and trajectory evidence suggests the student learns format over strategy. Confirming the dominant cause requires further experiments.

Why pure RL stalls but helps search. Standalone GRPO fails for a clear mechanistic reason. GRPO normalizes advantages within each batch by the reward standard deviation σ_r. Early in training on ARC, all K rollouts typically fail, so $\sigma_r = 0$ and the gradient is skipped. Unlike ordinary sparse rewards, the policy receives *no signal at all*.

ARC solutions require composing 2–4 DSL primitives in a precise puzzle-specific order, but a random MLP assigns near-uniform probability over 87–203 actions with no visual bias. The chance of randomly generating a correct two-step program is roughly $|\mathcal{A}|^{-2}$, below 10^{-4}; across 104 puzzles with different compositions, expected successes per episode are negligible. This is not a failure of the reward function or algorithm: denser intermediate rewards mislead gradients because ARC intermediate states are compositional, not partial solutions. PPO or REINFORCE doesn't help either, as all policy-gradient methods require reward variance within a batch. The cold-start problem is thus intrinsic to the task.

Still, the partially learned policy encodes a weak but nonzero bias toward operators used in successful programs. When used as a prior in beam search, it effectively prunes the search tree, matching the DFS solved count at $6\times$ lower runtime. This suggests the correct inference strategy is constrained beam search guided by a high-quality operator prior. Improving that prior—e.g., via behavioral cloning warm-start—offers the highest-leverage path forward.

7 Limitations and Future Work

Evaluation pools differ (85 for API vs. 27 for SFT/RL), limiting direct comparison. SFT remains limited ($\sim$15% cell accuracy, 0 solves even with gpt-oss-120b distillation) and merits further study. Prompt-level compliance controls remain leaky and should be replaced by grammar-constrained decoding.

Future work should prioritize: (1) larger and more diverse demonstrations; (2) behavior-cloning warm-start before RL; (3) strict structured decoding over valid DSL syntax; and (4) parameter-efficient adaptation for faster ablation cycles.

8 Conclusion

We presented a constrained DSL environment for ARC-AGI that enables clean comparison of API LLM, SFT, and RL approaches. Results indicate that direct end-to-end training is currently limited by data scarcity and reward sparsity, while strong LLM planners excel under executable action constraints. Most importantly, RL becomes valuable when used as a search prior rather than a standalone solver. These findings support a hybrid recipe for ARC: symbolic structure for reliability, learned priors for efficiency, and explicit verification for correctness.

References

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

François Chollet, Greg Kamradt (Kaifishr), Greg Knoop, Bryan Landers, and Nihar Udupa. Arc-agi: A new benchmark for frontier intelligence. *arXiv preprint arXiv:2407.04620*, 2024.

Thomas Hodel. Addressing the abstraction and reasoning corpus via procedural example generation. *arXiv preprint arXiv:2404.07353*, 2024.

Thomas Hodel. arc-dsl: Domain-specific language for abstraction and reasoning corpus tasks. GitHub repository, 2023. `https://github.com/michaelhodel/arc-dsl`.

Sven Le Gris, Cameron Jones, and Brenden M. Lake. H-arc: A robust estimate of human performance on the abstraction and reasoning corpus benchmark. *Scientific Data*, 12(126):1–15, 2025.

Felix Lee, Max Dabagia, and Fabian Monrose. Conceptarc: A benchmark for evaluating understanding and generalization in the abstraction and reasoning corpus. *arXiv preprint arXiv:2310.05146*, 2023.

Zhe Wei Tan, Danyang Wang, and Ruibo Liu. Seearc: A vision-centric benchmark for reasoning and abstraction in language models. *arXiv preprint arXiv:2403.11793*, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Matt Ho, Jason Wei, Hyung Won Chung, Huy Nguyen, and Adam Kalai. ArcMemo: Towards learning-based test-time scaling for ARC-AGI. *arXiv preprint arXiv:2509.04439*, 2025.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Zaid Hammoud, Luke Yang, Jiachang Mei, and Teven Le Scao. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.